

```

import numpy as np
from dataclasses import dataclass
from typing import Tuple, List, Dict
from collections import defaultdict
import itertools
import scipy.sparse as sp
import math
import scipy.linalg as la
import scipy.sparse.linalg as sla

@dataclass
class TensorTerm:
    tensor: np.ndarray
    daggers: Tuple[int, ...]

class Superposition:
    """An object-oriented wrapper for a linear combination of Fock states."""
    def __init__(self, state_dict=None):
        self.states = defaultdict(complex, state_dict if state_dict else {})

    def __add__(self, other, tol=1e-12):
        result = self.states.copy()
        for st, amp in other.states.items():
            result[st] += amp
        return Superposition(self._clean_dict(result, tol=tol))

    def __rmul__(self, scalar, tol=1e-12):
        result = {st: scalar * amp for st, amp in self.states.items()}
        return Superposition(self._clean_dict(result, tol=tol))

    def _clean_dict(self, d, tol=1e-12):
        return {st: amp for st, amp in d.items() if abs(amp) > tol}

    def normalize(self):
        norm = np.sqrt(sum(abs(amp)**2 for amp in self.states.values()))
        if norm > 0:
            self.states = {st: amp / norm for st, amp in self.states.items()}
        return self

    def trim(self, threshold=0.99):
        self.normalize()
        sorted_items = sorted(self.states.items(), key=lambda item: abs(item[1])**2, reverse=True)
        trimmed_state = {}
        accumulated_prob = 0.0

```

```

        for st, amp in sorted_items:
            trimmed_state[st] = amp
            accumulated_prob += abs(amp)**2
            if accumulated_prob >= threshold:
                break

        self.states = trimmed_state
        return self.normalize()

class QuantumOperator:
    """An algebraically aware container for tensor terms."""
    def __init__(self, terms=None):
        self.terms = terms if terms is not None else []

    def __add__(self, other):
        return QuantumOperator(self.terms + other.terms)

    def __rmul__(self, scalar):
        new_terms = [TensorTerm(term.tensor * scalar, term.daggers) for term in self.terms]
        return QuantumOperator(new_terms)

    def __mul__(self, other):
        """Computes the product of two operators via tensor outer product."""
        new_terms = []
        for tA in self.terms:
            for tB in other.terms:
                # np.tensordot with axes=0 computes the full outer product
                new_tensor = np.tensordot(tA.tensor, tB.tensor, axes=0)
                new_daggers = tA.daggers + tB.daggers
                new_terms.append(TensorTerm(new_tensor, new_daggers))
        return QuantumOperator(new_terms)

    def simplify(self, tol: float = 1e-12):
        """Optimizes the operator by merging tensors with identical dagger structures."""
        accumulated = {}
        for term in self.terms:
            if term.daggers not in accumulated:
                accumulated[term.daggers] = np.zeros_like(term.tensor)
            accumulated[term.daggers] += term.tensor

        self.terms = [
            TensorTerm(tensor, dags)
            for dags, tensor in accumulated.items()
            if np.any(np.abs(tensor) > tol) # Drop pure noise tensors
        ]

```

```

        return self

class SecondQuantizationEngine:
    def __init__(self, n_modes: int, statistics: str = 'boson',
                  coeff_tol: float = 1e-9, zero_tol: float = 1e-12):
        if statistics not in ['boson', 'fermion']:
            raise ValueError("Statistics must be 'boson' or 'fermion'")
        self.n_modes = n_modes
        self.statistics = statistics
        self.sign = 1 if statistics == 'boson' else -1
        # coeff_tol: threshold for pruning matrix-element/amplitude magnitudes
        # (Fock-space construction, operator application). Matches the original
        # single-file implementation's 1e-9 gate on every add().
        self.coeff_tol = coeff_tol
        # zero_tol: threshold for structural "is this tensor exactly zero"
        # decisions (normal ordering, basis-change branch pruning). Distinct
        # from coeff_tol because it's answering a different question.
        self.zero_tol = zero_tol

    def _get_parity_phase(self, mode: int, state: Tuple[int, ...]) -> int:
        """Calculates the Jordan-Wigner phase for fermions."""
        if self.statistics == 'boson':
            return 1
        # For fermions, count particles in modes prior to the target mode
        particles_before = sum(state[:mode])
        return -1 if particles_before % 2 == 1 else 1

    def _apply_annihilation(self, mode: int, state: Tuple[int, ...]) -> Tuple[Tuple[int, ...], float]:
        n = state[mode]
        if n == 0:
            return None, 0.0 # Annihilating the vacuum yields 0

        phase = self._get_parity_phase(mode, state)
        new_state = list(state)
        new_state[mode] -= 1

        amplitude = phase * np.sqrt(n)
        return tuple(new_state), amplitude

    def _apply_creation(self, mode: int, state: Tuple[int, ...]) -> Tuple[Tuple[int, ...], float]:
        n = state[mode]
        if self.statistics == 'fermion' and n == 1:
            return None, 0.0 # Pauli exclusion principle

        phase = self._get_parity_phase(mode, state)

```

```

new_state = list(state)
new_state[mode] += 1

amplitude = phase * np.sqrt(n + 1)
return tuple(new_state), amplitude

def apply_operator(self, operator: QuantumOperator, superposition: Superposition) -> Superposition:
    """Updated to interface seamlessly with the new Superposition class."""
    result_state = defaultdict(complex)
    active_states = superposition._clean_dict(superposition.states, tol=self.coeff_tol)

    if not active_states:
        return Superposition()

    for term in operator.terms:
        non_zero_indices = np.argwhere(np.abs(term.tensor) > self.coeff_tol)
        for idx in non_zero_indices:
            idx_tuple = tuple(idx)
            coeff = term.tensor[idx_tuple]

            current_branches = {st: amp * coeff for st, amp in active_states.items()}

            for i in reversed(range(len(term.daggers))):
                mode = idx_tuple[i]
                is_dagger = term.daggers[i]
                next_branches = defaultdict(complex)

                for curr_state, curr_amp in current_branches.items():
                    if is_dagger:
                        new_st, amp_factor = self._apply_creation(mode, curr_state)
                    else:
                        new_st, amp_factor = self._apply_annihilation(mode, curr_state)

                    if new_st is not None:
                        next_branches[new_st] += curr_amp * amp_factor

                current_branches = next_branches
                if not current_branches:
                    break

            for final_st, final_amp in current_branches.items():
                result_state[final_st] += final_amp

    return Superposition(result_state)

def normalize_state(self, superposition: Dict[Tuple[int, ...], complex]) -> Dict[Tuple[int, ...], complex]:

```

```

    """
    Normalizes a linear combination of Fock states so that the sum of  $|c|^2$  is 1.
    """
    # Calculate the norm:  $\sqrt{\sum |c|^2}$ 
    norm_sq = sum(abs(amp)**2 for amp in superposition.values())

    if np.isclose(norm_sq, 0.0):
        return {} # Cannot normalize the vacuum/zero-vector

    norm = np.sqrt(norm_sq)
    return {st: amp / norm for st, amp in superposition.items()}

def trim_state(self, superposition: Dict[Tuple[int, ...], complex], threshold: float = 0.0)
    """
    Normalizes, sorts by magnitude descending, and truncates to keep the most
    significant terms that sum up to the probability threshold.
    """
    # Step 1: Normalize the incoming state
    normalized_state = self.normalize_state(superposition)

    # Step 2: Sort descending by absolute magnitude squared ( $|c|^2$ )
    sorted_items = sorted(normalized_state.items(), key=lambda item: abs(item[1])**2, reverse=True)

    # Step 3: Accumulate probabilities and trim
    trimmed_state = {}
    accumulated_prob = 0.0

    for st, amp in sorted_items:
        prob = abs(amp)**2
        trimmed_state[st] = amp
        accumulated_prob += prob

        # Stop once the accumulated probability hits or exceeds the threshold
        if accumulated_prob >= threshold:
            break

    # Step 4: Re-normalize the trimmed state so it is completely unitary again
    return self.normalize_state(trimmed_state)

def normal_order(self, operator: QuantumOperator) -> QuantumOperator:
    """
    Reduces a QuantumOperator to normal order using Wick's theorem.
    Returns a newly optimized QuantumOperator object.
    """
    pending_terms = operator.terms.copy()
    accumulated_terms = {}

```

```

while pending_terms:
    term = pending_terms.pop(0)
    daggers = term.daggers
    tensor = np.asarray(term.tensor)

    swap_idx = -1
    for i in range(len(daggers) - 1):
        if daggers[i] == 0 and daggers[i+1] == 1:
            swap_idx = i
            break

    if swap_idx != -1:
        # Swapped Term
        new_daggers_swap = list(daggers)
        new_daggers_swap[swap_idx], new_daggers_swap[swap_idx+1] = 1, 0
        new_tensor_swap = np.swapaxes(tensor, swap_idx, swap_idx+1) * self.sign
        pending_terms.append(TensorTerm(new_tensor_swap, tuple(new_daggers_swap)))

        # Contracted Term
        new_daggers_contract = tuple(daggers[:swap_idx] + daggers[swap_idx+2:])
        new_tensor_contract = np.trace(tensor, axis1=swap_idx, axis2=swap_idx+1)
        pending_terms.append(TensorTerm(new_tensor_contract, new_daggers_contract))
    else:
        if daggers not in accumulated_terms:
            accumulated_terms[daggers] = np.zeros_like(tensor)
            accumulated_terms[daggers] += tensor

final_terms = [
    TensorTerm(tensor, dags)
    for dags, tensor in accumulated_terms.items()
    if not np.allclose(tensor, 0, atol=self.zero_tol)
]
return QuantumOperator(final_terms)

def change_basis(self, operator: QuantumOperator, U: np.ndarray, V: np.ndarray) -> QuantumOperator
    """
    Applies a Bogoliubov transformation to the operator.
    
$$b_i = U_{ij} b_j + V_{ij} b_{\text{dag}_j}^{\dagger}$$

    """
    new_terms = []

    for original_term in operator.terms:
        k = len(original_term.daggers)

        # Start the branching with the current tensor and daggers

```

```

# We use lists for daggers temporarily so we can mutate them during branching
branches = [(original_term.tensor, list(original_term.daggers))]

for axis in range(k):
    next_branches = []

    for tensor, daggers in branches:
        is_dagger = daggers[axis]

        if is_dagger == 0:
            # Annihilation transforms:  $b_i = U_{ij} b_j + V_{ij} b^{\dagger}_j$ 

            # Branch 1: U (annihilation)
            t_U = np.tensordot(tensor, U, axes=([axis], [0]))
            t_U = np.moveaxis(t_U, -1, axis)
            dag_U = daggers.copy()
            dag_U[axis] = 0
            next_branches.append((t_U, dag_U))

            # Branch 2:  $V^*$  (creation)
            t_V_conj = np.tensordot(tensor, np.conj(V), axes=([axis], [0]))
            t_V_conj = np.moveaxis(t_V_conj, -1, axis)
            dag_V = daggers.copy()
            dag_V[axis] = 1
            next_branches.append((t_V_conj, dag_V))

        else:
            # Creation transforms:  $b^{\dagger}_i = U_{ij} b^{\dagger}_j + V_{ij} b_j$ 

            # Branch 1:  $U^*$  (creation)
            t_U_conj = np.tensordot(tensor, np.conj(U), axes=([axis], [0]))
            t_U_conj = np.moveaxis(t_U_conj, -1, axis)
            dag_U_conj = daggers.copy()
            dag_U_conj[axis] = 1
            next_branches.append((t_U_conj, dag_U_conj))

            # Branch 2: V (annihilation)
            t_V = np.tensordot(tensor, V, axes=([axis], [0]))
            t_V = np.moveaxis(t_V, -1, axis)
            dag_V = daggers.copy()
            dag_V[axis] = 0
            next_branches.append((t_V, dag_V))

    # Update branches for the next axis
    branches = next_branches

```

```

        # After iterating all axes, commit the 2^k branches to the new operator
        for tensor, dags in branches:
            if not np.allclose(tensor, 0, atol=self.zero_tol): # Filter out exact zeros
                new_terms.append(TensorTerm(tensor, tuple(dags)))

    return QuantumOperator(new_terms)

def generate_basis(self, n_excitations: int) -> List[Tuple[int, ...]]:
    """
    Generates all valid Fock states for the system with exactly `n_excitations`.
    Automatically handles Bosonic vs Fermionic statistics.
    """
    if n_excitations < 0:
        return []

    if self.statistics == 'fermion':
        if n_excitations > self.n_modes:
            return [] # Pauli exclusion: cannot have more fermions than modes

        # For fermions, we simply choose `n_excitations` modes to occupy
        basis = []
        for occupied_indices in itertools.combinations(range(self.n_modes), n_excitations):
            state = [0] * self.n_modes
            for idx in occupied_indices:
                state[idx] = 1
            basis.append(tuple(state))
        return basis

    else:
        # For bosons, we use the recursive stars-and-bars integer partitioning
        def compositions(n, total):
            if n == 1:
                yield (total,)
                return
            for first in range(total + 1):
                for rest in compositions(n - 1, total - first):
                    yield (first,) + rest

        return list(compositions(self.n_modes, n_excitations))

def build_operator_matrix(self, operator: QuantumOperator, basis: List[Tuple[int, ...]]):
    """
    Computes the dense matrix representation of an operator within a given basis.
    Rows correspond to the 'bra' (output state), Columns correspond to the 'ket' (input state)
    """
    dim = len(basis)

```



```

matrix = np.zeros((dim, dim), dtype=complex)

# Create a fast lookup dictionary for the row index of each resulting state
state_to_index = {state: idx for idx, state in enumerate(basis)}

# Iterate over the basis (Columns/Kets)
for col_idx, ket_state in enumerate(basis):
    # Formulate the ket as a superposition dictionary
    # Wrap the dict in the new Superposition class
    initial_superposition = Superposition({ket_state: 1.0 + 0j})

    # Apply the operator to the ket
    result_superposition = self.apply_operator(operator, initial_superposition)

    # Call .items() on the .states dictionary, not the Superposition object itself
    for result_state, amplitude in result_superposition.states.items():
        if result_state in state_to_index:
            row_idx = state_to_index[result_state]
            matrix[row_idx, col_idx] += amplitude

return matrix

def build_sparse_operator_matrix(self, operator: QuantumOperator, basis: List[Tuple[int, str]]):
    """
    Computes the sparse matrix representation of an operator within a given basis.
    Returns a CSR format sparse matrix optimized for fast arithmetic and diagonalization.
    """
    dim = len(basis)

    # DOK (Dictionary of Keys) format is the fastest for constructing sparse matrices
    # when you are adding elements one by one.
    matrix = sp.dok_matrix((dim, dim), dtype=complex)

    # Fast lookup dictionary
    state_to_index = {state: idx for idx, state in enumerate(basis)}

    # Iterate over the basis (Columns/Kets)
    for col_idx, ket_state in enumerate(basis):
        # Wrap the dict in the new Superposition class
        initial_superposition = Superposition({ket_state: 1.0 + 0j})

        # Apply the operator to the ket
        result_superposition = self.apply_operator(operator, initial_superposition)

        # Call .items() on the .states dictionary, not the Superposition object itself
        for result_state, amplitude in result_superposition.states.items():

```

```

        if abs(amplitude) <= self.coeff_tol:
            continue
        if result_state in state_to_index:
            row_idx = state_to_index[result_state]
            matrix[row_idx, col_idx] += amplitude

# Convert to CSR format before returning.
# CSR is required for fast Scipy linear algebra operations (like eigsh).
return matrix.tocsr()

def build_rectangular_operator_matrix(self, operator: QuantumOperator,
                                     ket_basis: List[Tuple[int, ...]],
                                     bra_basis: List[Tuple[int, ...]]) -> sp.csr_matrix:
    """
    Computes the matrix of `operator` mapping ket_basis -> bra_basis when
    the two live in different excitation sectors (e.g. a creation-pair
    operator taking an n-excitation state to an (n+2)-excitation one).
    build_sparse_operator_matrix assumes a single shared basis for both
    rows and columns, which doesn't apply here -- this is the rectangular
    generalization, built the same way (apply_operator per ket column).
    """
    n_cols = len(ket_basis)
    n_rows = len(bra_basis)
    matrix = sp.dok_matrix((n_rows, n_cols), dtype=complex)
    state_to_row = {state: idx for idx, state in enumerate(bra_basis)}

    for col_idx, ket_state in enumerate(ket_basis):
        initial_superposition = Superposition({ket_state: 1.0 + 0j})
        result_superposition = self.apply_operator(operator, initial_superposition)
        for result_state, amplitude in result_superposition.states.items():
            if abs(amplitude) <= self.coeff_tol:
                continue
            if result_state in state_to_row:
                matrix[state_to_row[result_state], col_idx] += amplitude

    return matrix.tocsr()

def build_fast_sparse_matrix(self, operator: QuantumOperator, basis: list) -> sp.csr_matrix:
    """Massively optimized sparse matrix builder using COO format."""
    dim = len(basis)
    state_to_index = {state: idx for idx, state in enumerate(basis)}

    operator.simplify() # Ensure redundant tensors are merged before loop

    rows, cols, data = [], [], []

```

```

for col_idx, ket_state in enumerate(basis):
    initial_super = Superposition({ket_state: 1.0 + 0j})
    result_super = self.apply_operator(operator, initial_super)

    for result_state, amplitude in result_super.states.items():
        if abs(amplitude) <= self.coeff_tol:
            continue
        if result_state in state_to_index:
            rows.append(state_to_index[result_state])
            cols.append(col_idx)
            data.append(amplitude)

# Construct the COO matrix in a single vectorized burst, then convert to CSR
coo = sp.coo_matrix((data, (rows, cols)), shape=(dim, dim), dtype=complex)
return coo.tocsr()

def exp_operator(self, operator: QuantumOperator, order: int) -> QuantumOperator:
    """
    Computes the Taylor expansion  $\exp(A) = I + A + A^2/2! + \dots + A^n/n!$ 
    and returns the fully normal-ordered QuantumOperator.
    """
    # Identity operator: 0-order tensor, scalar 1.0
    identity_term = TensorTerm(np.array(1.0 + 0j), ())
    result_op = QuantumOperator([identity_term])

    if order == 0:
        return result_op

    current_power = operator
    result_op = result_op + (1.0 * current_power)

    for n in range(2, order + 1):
        # Multiply operator by itself to get next power
        current_power = current_power * operator

        # Crucial optimization: simplify before the next multiplication
        # to prevent the tensor array sizes from bottlenecking RAM
        current_power.simplify()

        term_to_add = (1.0 / math.factorial(n)) * current_power
        result_op = result_op + term_to_add

    result_op.simplify()

    # Pass the final summation through the Wick's theorem engine
    return self.normal_order(result_op)

```

```

def _extract_quadratic_blocks(self, operator: QuantumOperator) -> Tuple[np.ndarray, np.ndarray]:
    """Extracts the A ( $b^\dagger b$ ) and B ( $b^\dagger b^\dagger$ ) matrices from a quadratic operator."""
    operator.simplify()
    is_real = all(np.isrealobj(term.tensor) for term in operator.terms)
    dtype = np.float64 if is_real else complex
    A = np.zeros((self.n_modes, self.n_modes), dtype=dtype)
    B = np.zeros((self.n_modes, self.n_modes), dtype=dtype)

    for term in operator.terms:
        if len(term.daggers) != 2:
            raise ValueError("Operator contains non-quadratic terms.")

        if term.daggers == (1, 0): #  $b^\dagger_i b_j$ 
            A += term.tensor
        elif term.daggers == (1, 1): #  $b^\dagger_i b^\dagger_j$ 
            B += term.tensor

    return A, B

def diagonalize_quadratic(self, operator: QuantumOperator):
    # ... (extract A and B)
    A, B = self._extract_quadratic_blocks(operator)
    n = self.n_modes

    # FIX: Multiply B by 2 to align the Van Hemmen 1/2 factor
    # with standard tensor summation conventions
    B_scaled = 2.0 * B

    is_real = np.isrealobj(A) and np.isrealobj(B_scaled)
    dtype = np.float64 if is_real else complex

    # Build the Hermitian dynamical matrix. Conjugation is a no-op for
    # real input but kept dtype-aware: forcing complex unconditionally
    # here would route real problems through a different LAPACK routine
    # (zheevd vs dsyevd) than the original hand-rolled implementation,
    # which has no guaranteed agreement on eigenvector sign for
    # degenerate/near-degenerate clusters.
    D_herm = np.block([
        [A, B_scaled],
        [B_scaled.conj().T, A.conj()]
    ]).astype(dtype)

    tau_z = np.block([
        [np.eye(n), np.zeros((n, n))],
        [np.zeros((n, n)), -np.eye(n)]
    ])

```

```

]).astype(dtype)

# Van Hemmen Cholesky decomposition
L = np.linalg.cholesky(D_herm)
M_sym = L.T @ tau_z @ L

evals_sym, evecs_sym = la.eigh(M_sym)
evecs_real = la.solve_triangular(L.T, evecs_sym)

# Keep positive branch
pos_idx = evals_sym > self.zero_tol
pos_evals = evals_sym[pos_idx]
pos_evecs = evecs_real[:, pos_idx]

sort_idx = np.argsort(pos_evals)
pos_evals = pos_evals[sort_idx]
pos_evecs = pos_evecs[:, sort_idx]

# Symplectic Gram-Schmidt
def symplectic_gram_schmidt(vecs):
    n_cols = vecs.shape[1]
    result = np.zeros_like(vecs, dtype=vecs.dtype)
    for i in range(n_cols):
        v = vecs[:, i].copy()
        for j in range(i):
            e_j = result[:, j]
            proj = e_j.conj().T @ tau_z @ v
            v = v - proj * e_j
        norm = np.real(v.conj().T @ tau_z @ v)
        result[:, i] = v / np.sqrt(norm)
    return result

U_V = symplectic_gram_schmidt(pos_evecs)

# Canonicalize sign: force the largest-magnitude component of each
# column to be positive (real part, for the real-valued case). LAPACK
# only guarantees eigenspaces up to sign/phase, not a specific choice,
# so different call sites (or different dtypes/routines) can
# otherwise disagree on sign for quantities that are odd in it
# (e.g. the B_J/C_J pairing blocks used downstream).
for col in range(U_V.shape[1]):
    v = U_V[:, col]
    idx = np.argmax(np.abs(v))
    ref = v[idx]
    if np.real(ref) < 0 or (np.real(ref) == 0 and np.imag(ref) < 0):
        U_V[:, col] = -v

```

```

U = U_V[:n, :]
V = U_V[n:, :]

return pos_evals, U, V

def build_squeezed_vacuum(self, U: np.ndarray, V: np.ndarray, truncation: int, threshold: float):
    """
    Generates the exact analytical squeezed vacuum state for a quadratic Hamiltonian.
    """
    n = self.n_modes

    # RECONSTRUCT U_V HERE:
    U_V = np.vstack([U, V])

    # Build the full pseudo-unitary transformation matrix to find the inverse
    tau_x = np.block([[np.zeros((n, n)), np.eye(n)], [np.eye(n), np.zeros((n, n))]])
    J_U_V = tau_x @ U_V.conj() # Pseudo-time reversal for the negative branch
    P = np.hstack([U_V, J_U_V])
    P_inv = np.linalg.inv(P)

    U_tilde = P_inv[:n, :n]
    V_tilde = P_inv[n:, n:]
    M = np.linalg.solve(U_tilde, V_tilde)

    vacuum = tuple([0] * n)
    current_terms = {vacuum: 1.0 + 0j}

    final_state = defaultdict(complex)
    final_state[vacuum] = 1.0 + 0j

    for k in range(1, truncation + 1):
        next_terms = defaultdict(complex)
        for state_tuple, coeff in current_terms.items():
            state = list(state_tuple)
            for j in range(n):
                for i in range(n):
                    val = M[i, j]
                    if abs(val) < self.coeff_tol:
                        continue

                    new_state = state.copy()
                    new_state[j] += 1
                    factor_j = np.sqrt(new_state[j])
                    new_state[i] += 1
                    factor_i = np.sqrt(new_state[i])

```

```

        next_terms[tuple(new_state)] += coeff * (-0.5) * val * factor_j * fa

    for st, val in next_terms.items():
        if abs(val) > self.coeff_tol:
            final_state[st] += val
    current_terms = next_terms

    return Superposition(final_state).trim(threshold)

def transform_state(self, state: Superposition, U: np.ndarray, V: np.ndarray,
                    direction: str = 'old_to_new',
                    base_state: 'Superposition' = None) -> Superposition:
    """
    Transforms a state vector between the bare Fock basis and the Bogoliubov
    eigenbasis, by repeated application of a single quasiparticle creation
    operator (built from rows of U, V) to a base/reference state.

    direction='old_to_new': U, V are used as-given (rows = per-mode
    coefficients), and the base state defaults to the analytic squeezed
    vacuum built from U, V.

    direction='new_to_old' (or anything else): U, V are used as-given too
    -- the caller is responsible for passing whatever U, V already
    correctly encode the inverse transform (e.g.  $P[:,n,:n].T$ ,  $-P[n,:n].T$ 
    for a real pseudo-unitary  $P$ ), since re-deriving  $P^{-1}$  from U, V here
    assumes a specific complex Bogoliubov convention ( $P = [[U, V^*], [V, U^*]]$ )
    that does not match every caller's P construction. The base state
    defaults to the bare vacuum.

    base_state: optional explicit starting Superposition (e.g. a
    precomputed squeezed vacuum) to use instead of either default. This
    is what lets callers reuse a cached vacuum (self.vacuum,
    self.unsqueezed_vacuum) instead of recomputing it via
    build_squeezed_vacuum on every call.

    Returns the untrimmed result (only the 1e-12 numerical-noise cleanup
    from Superposition arithmetic is applied). Callers that want a
    threshold-based truncation should call trim_state()/trim() explicitly
    with their own threshold -- this method no longer trims internally,
    since a hardcoded internal threshold silently overrides whatever
    threshold the caller actually wants, and double-trimming (once here,
    once by the caller) corrupts the result.
    """
    n = self.n_modes
    U_mat, V_mat = U, V

```

```

if base_state is not None:
    base_vacuum = base_state
elif direction == 'old_to_new':
    base_vacuum = self.build_squeezed_vacuum(U, V, truncation=10) # Approximated bas
else:
    base_vacuum = Superposition({tuple([0] * n): 1.0 + 0j})

result = Superposition()

for st_tuple, amp in state.states.items():
    occupations = np.array(st_tuple)
    current_branch = base_vacuum

    for mode_idx in range(n):
        r_i = occupations[mode_idx]
        if r_i == 0:
            continue

        for k in range(1, r_i + 1):
            # Apply the Bogoliubov transformed creation operator
            next_branch = defaultdict(complex)
            for br_st, br_amp in current_branch.states.items():
                # U component (Creation)
                for j in range(n):
                    if abs(U_mat[mode_idx, j]) > self.coeff_tol:
                        new_st, factor = self._apply_creation(j, br_st)
                        if new_st:
                            next_branch[new_st] += br_amp * U_mat[mode_idx, j] * fac
            # V component (Annihilation)
            for j in range(n):
                if abs(V_mat[mode_idx, j]) > self.coeff_tol:
                    new_st, factor = self._apply_annihilation(j, br_st)
                    if new_st:
                        next_branch[new_st] += br_amp * V_mat[mode_idx, j] * fac

            # Normalize factorial scaling
            current_branch = Superposition({s: v / np.sqrt(k) for s, v in next_branch.items()})

    result = result + (amp * current_branch)

return result

def build_quadratic_sparse_matrix(self, operator: QuantumOperator, basis: list) -> sp.csr_matrix:
    """
    Highly optimized sparse matrix builder specifically for quadratic Hamiltonians.
    Bypasses generalized tensor contractions for massive speedup.

```



```

"""
A, B = self._extract_quadratic_blocks(operator)
n = self.n_modes
dim = len(basis)
state_to_index = {state: idx for idx, state in enumerate(basis)}

rows, cols, data = [], [], []

def add_element(r, c, val):
    if abs(val) > self.coeff_tol:
        rows.append(r)
        cols.append(c)
        data.append(val)

for col_idx, ket in enumerate(basis):
    ket_arr = np.array(ket)

    # Number-conserving terms: A (b^dag_i b_j)
    for j in range(n):
        if ket_arr[j] == 0:
            continue
        for i in range(n):
            if abs(A[i, j]) < self.coeff_tol:
                continue
            new_st = ket_arr.copy()
            new_st[j] -= 1
            factor_j = np.sqrt(ket_arr[j])
            factor_i = np.sqrt(new_st[i] + 1)
            new_st[i] += 1

            st_tuple = tuple(new_st)
            if st_tuple in state_to_index:
                add_element(state_to_index[st_tuple], col_idx, A[i, j] * factor_i *

    # Pairing terms: B (b^dag_i b^dag_j)
    for j in range(n):
        for i in range(n):
            if abs(B[i, j]) < self.coeff_tol:
                continue
            new_st = ket_arr.copy()
            new_st[j] += 1
            factor_j = np.sqrt(new_st[j])
            new_st[i] += 1
            factor_i = np.sqrt(new_st[i])

            st_tuple = tuple(new_st)

```

```

        if st_tuple in state_to_index:
            add_element(state_to_index[st_tuple], col_idx, B[i, j] * factor_i *

# Conjugate Pairing terms: B^* (b_i b_j)
B_conj = B.conj().T
for j in range(n):
    if ket_arr[j] == 0:
        continue
    for i in range(n):
        if abs(B_conj[i, j]) < self.coeff_tol:
            continue
        new_st = ket_arr.copy()
        factor_j = np.sqrt(new_st[j])
        new_st[j] -= 1
        if new_st[i] == 0:
            continue
        factor_i = np.sqrt(new_st[i])
        new_st[i] -= 1

        st_tuple = tuple(new_st)
        if st_tuple in state_to_index:
            add_element(state_to_index[st_tuple], col_idx, B_conj[i, j] * factor

return sp.coo_matrix((data, (rows, cols)), shape=(dim, dim), dtype=complex).tocsr()

def partial_trace(self, state: Superposition, keep_modes: List[int]) -> np.ndarray:
    """
    Computes the reduced density matrix by tracing out all modes NOT in `keep_modes`.
    """
    keep_modes = sorted(keep_modes)
    trace_modes = [i for i in range(self.n_modes) if i not in keep_modes]

    # Determine the dimension of the kept subspace dynamically based on the state
    max_excitations = {m: 0 for m in keep_modes}
    for st in state.states.keys():
        for m in keep_modes:
            max_excitations[m] = max(max_excitations[m], st[m])

    # Create an index mapping for the kept basis
    import itertools
    ranges = [range(max_excitations[m] + 1) for m in keep_modes]
    kept_basis = list(itertools.product(*ranges))
    kept_idx = {st: i for i, st in enumerate(kept_basis)}
    dim = len(kept_basis)

    rho_reduced = np.zeros((dim, dim), dtype=complex)

```

```

# Group states by their traced-out mode configuration
trace_groups = defaultdict(list)
for st_tuple, amp in state.states.items():
    traced_config = tuple(st_tuple[m] for m in trace_modes)
    kept_config = tuple(st_tuple[m] for m in keep_modes)
    trace_groups[traced_config].append((kept_config, amp))

# Perform the partial trace: sum over outer products of matching traced configurations
for traced_config, kept_components in trace_groups.items():
    for (kept_1, amp_1) in kept_components:
        for (kept_2, amp_2) in kept_components:
            i = kept_idx[kept_1]
            j = kept_idx[kept_2]
            rho_reduced[i, j] += amp_1 * np.conj(amp_2)

return rho_reduced

def get_mode_occupations(self, state: Superposition) -> np.ndarray:
    """Calculates the expected photon/excitation number <n_i> for each mode."""
    occupations = np.zeros(self.n_modes, dtype=float)
    for st_tuple, amp in state.states.items():
        prob = abs(amp)**2
        for i in range(self.n_modes):
            occupations[i] += st_tuple[i] * prob
    return occupations

def get_total_excitations(self, state: Superposition) -> float:
    """Returns the total expected number of excitations in the system."""
    return np.sum(self.get_mode_occupations(state))

class CompositeEngine:
    """A general-purpose engine for bipartite tensor-product Hilbert spaces H_A H_B."""

    def vector_to_subsystems(self, full_vector: np.ndarray, basis_A: list, basis_B: list, tol: float):
        """
        Maps a 1D state vector of H_A H_B into a dictionary of {st_A: Superposition_B}.
        """
        dim_A = len(basis_A)
        dim_B = len(basis_B)

        if full_vector.shape[0] != dim_A * dim_B:
            raise ValueError(f"Vector size {full_vector.shape[0]} does not match basis dimensions")

        reshaped = full_vector.reshape((dim_A, dim_B))

```

```

state_dict = {}

for i, st_A in enumerate(basis_A):
    matter_dict = {}
    for j, st_B in enumerate(basis_B):
        amp = reshaped[i, j]
        if abs(amp) > threshold:
            matter_dict[st_B] = amp
    if matter_dict:
        state_dict[st_A] = Superposition(matter_dict)

return state_dict

def subsystems_to_vector(self, state_dict: dict, basis_A: list, basis_B: list) -> np.ndarray:
    """
    Flattens a dictionary of {st_A: Superposition_B} back into a 1D complex numpy array.
    """
    dim_A = len(basis_A)
    dim_B = len(basis_B)

    # Fast reverse lookups
    idx_A = {st: i for i, st in enumerate(basis_A)}
    idx_B = {st: i for i, st in enumerate(basis_B)}

    psi = np.zeros((dim_A, dim_B), dtype=complex)

    for st_A, superpos in state_dict.items():
        if st_A not in idx_A:
            continue
        i = idx_A[st_A]
        for st_B, amp in superpos.states.items():
            if st_B in idx_B:
                j = idx_B[st_B]
                psi[i, j] += amp

    return psi.reshape(dim_A * dim_B)

def map_subsystem_B(self, state_dict: dict, map_func) -> dict:
    """
    Applies a mapping function (e.g., Bogoliubov transforms) to the Subsystem B Superpos
    """
    new_dict = {}
    for st_A, superpos_B in state_dict.items():
        mapped_superpos = map_func(superpos_B)
        if mapped_superpos.states: # Only store if not empty
            new_dict[st_A] = mapped_superpos

```

```

        return new_dict

def partial_trace(self, full_vector: np.ndarray, dim_A: int, dim_B: int, keep: str = 'B')
    """
    Computes the reduced density matrix by tracing out one subsystem.
    """
    psi = full_vector.reshape(dim_A, dim_B)
    if keep == 'B':
        return psi.conj().T @ psi
    elif keep == 'A':
        return psi @ psi.conj().T
    else:
        raise ValueError("keep must be 'A' or 'B'")

def embed_matrix_A(self, matrix_A: sp.csr_matrix, dim_B: int) -> sp.csr_matrix:
    I_B = sp.eye(dim_B, format='csr')
    return sp.kron(matrix_A, I_B, format='csr')

def embed_matrix_B(self, dim_A: int, matrix_B: sp.csr_matrix) -> sp.csr_matrix:
    I_A = sp.eye(dim_A, format='csr')
    return sp.kron(I_A, matrix_B, format='csr')

def kron_matrices(self, matrix_A: sp.csr_matrix, matrix_B: sp.csr_matrix) -> sp.csr_matrix:
    return sp.kron(matrix_A, matrix_B, format='csr')

```